

LogPulse AI - Incident Report

Generated : 2026-05-31 16:53:11 UTC

Trace ID : 99091d2b-c1ec-40cc-b656-a6c1905aa0b9

1 - Incident Summary

Incident ID : 99091d2b
Category : Memory
Severity : SeverityLevel.CRITICAL
Source : Kubernetes
Cache : MISS

2 - Raw Log

2026-05-30 18:22:11 ERROR kubelet: Pod worker-api-7d9f8b-xk2q in namespace production OOMKilled (container=worker-api) exceeded 256Mi limit

3 - Classification

Category : Memory
Severity : SeverityLevel.CRITICAL
Summary : Kubernetes pod killed for exceeding its memory limit.
Source : Kubernetes

4 - Diagnosis

Root Cause : The `worker-api` container's configured memory limit of 256Mi was insufficient for its operational memory requirements, leading to Kubernetes OOMKilling the pod.
Confidence : 95%
Reasoning : The raw log explicitly states the pod was OOMKilled for exceeding its 256Mi limit, which is directly supported by the classification and the highly confident retrieved playbook for 'Pod OOMKilled Due to Insufficient Memory Limit'.
Source Tier : llm
Used History : No
Used Playbook : Yes

5 - Proposed Solution

Explanation:

Gemini unavailable; remediation derived from playbook 'Pod OOMKilled Due to Insufficient Memory Limit' (RAG confidence=0.8038). Review the playbook steps and adapt commands to the live environment before execution.

Steps:

- 1. Analyze historical application memory usage patterns using monitoring tools (e.g., Prometheus/Grafana, `kubectl top pod`). Identify the peak memory consumption.
- 2. Increase the `resources.limits.memory` for the affected container in its Deployment, StatefulSet, or Pod definition. It is often good practice to increase `resources.requests.memory` proportionally to ensure proper scheduling.
- 3. Apply the updated Kubernetes manifest to trigger a rollout and redeploy the affected pods.
- 4. Monitor the new deployment to confirm stable memory usage and absence of OOMKilled events.
- 5. name: my-app-container

Commands:

```
$ kubectl top pod
$ Apply the updated Kubernetes manifest to trigger a rollout and redeploy the affected pods.
$ kubectl apply -f deployment.yaml
```

6 - Risk Assessment

Risk Level : RISKLEVEL.SAFE

Safe Commands (3):

- [OK] kubectl top pod
- [OK] Apply the updated Kubernetes manifest to trigger a rollout and redeploy the affected pods.
- [OK] kubectl apply -f deployment.yaml

Flagged Commands (0):

(none)

7 - Operator Decision

Decision : OPERATORDECISION.APPROVED

Executed At : 2026-05-31T13:53:10.850164+00:00

Sandbox Results:

Command : kubectl top pod

Exit : 0

Time : 0ms

Output:

NAME	CPU(cores)	MEMORY(bytes)
worker-api-7d9f8b-xk2q	245m	241Mi
payment-svc-5f4b9-zx1w	120m	64Mi
payment-svc-5f4b9-ab2c	95m	71Mi
cache-service-9d1c2-kp7q	50m	128Mi
cache-service-9d1c2-mn4r	48m	... [truncated]

Command : Apply the updated Kubernetes manifest to trigger a rollout and redeploy the affected pods.

Exit : 0

Time : 0ms

Output:

[simulation] applied <manifest> (no manifest state in simulator)

Command : kubectl apply -f deployment.yaml

Exit : 0

Time : 0ms

Output:

[simulation] applied deployment.yaml (no manifest state in simulator)

8 - Historical Context

Times Seen : 1

First occurrence -- no prior incidents.

9 - Incident Timeline

[13:53:10] WORKFLOW_COMPLETED -> Completed in 138180ms